Portfolio

양현준 stringju

iam@2tle.io ↗ GitHub ↗ Blog ↗

Experiences

2025.04 · 2025.12

성균관대학교 산학협력 프로젝트
주식회사 커리어노트

프롬프트 엔지니어링 · 백엔드 개발 ↗

- 이력서·채용공고 첨삭 기능 개발
- 하계집중근무 진행 (2025.06 ~ 2025.08)

2025.03 · 2025.04

한봄고등학교

클라우드컴퓨팅 직종 기능대회 강사

- AWS 기초 및 관련 문제 풀이

2024.04 · 2026.12

성균관대학교 시스템컨설턴트그룹

백엔드, 인프라 엔지니어 ↗

- 팀 블로그 운영 ↗
- 신규 쿠버네티스 클러스터 구축 ↗
- 멀티테넌시 전시회 사이트 백엔드 개발 ↗
- S-TOP 사이트 백엔드 개발 ↗

2023.02.06 · 2023.02.24

마이다스인(마이다스아이티)

겨울방학 인턴십, Android 개발

- 모바일 HR 서비스 개발 과제 진행

Projects

1. 전시회 백엔드
2. Kubernetes One Repo
3. pi-provider-manager
4. S-TOP
5. 돌핀 (dolpin)

백엔드

전시회 백엔드

멀티테넌시를 지원하는 통합 온라인 전시회 백엔드입니다.

개발 기간 2026.01 ~

Java, Spring Boot, MySQL, MinIO

저장소 보기 ↗

프로젝트 소개

멀티테넌시를 지원하는 온라인 전시회 사이트의 백엔드입니다. 성균관대학교 시스템컨설턴트그룹에서 개발한 프로젝트로, 전반적인 프로젝트 진행과 백엔드 개발을 담당했습니다.

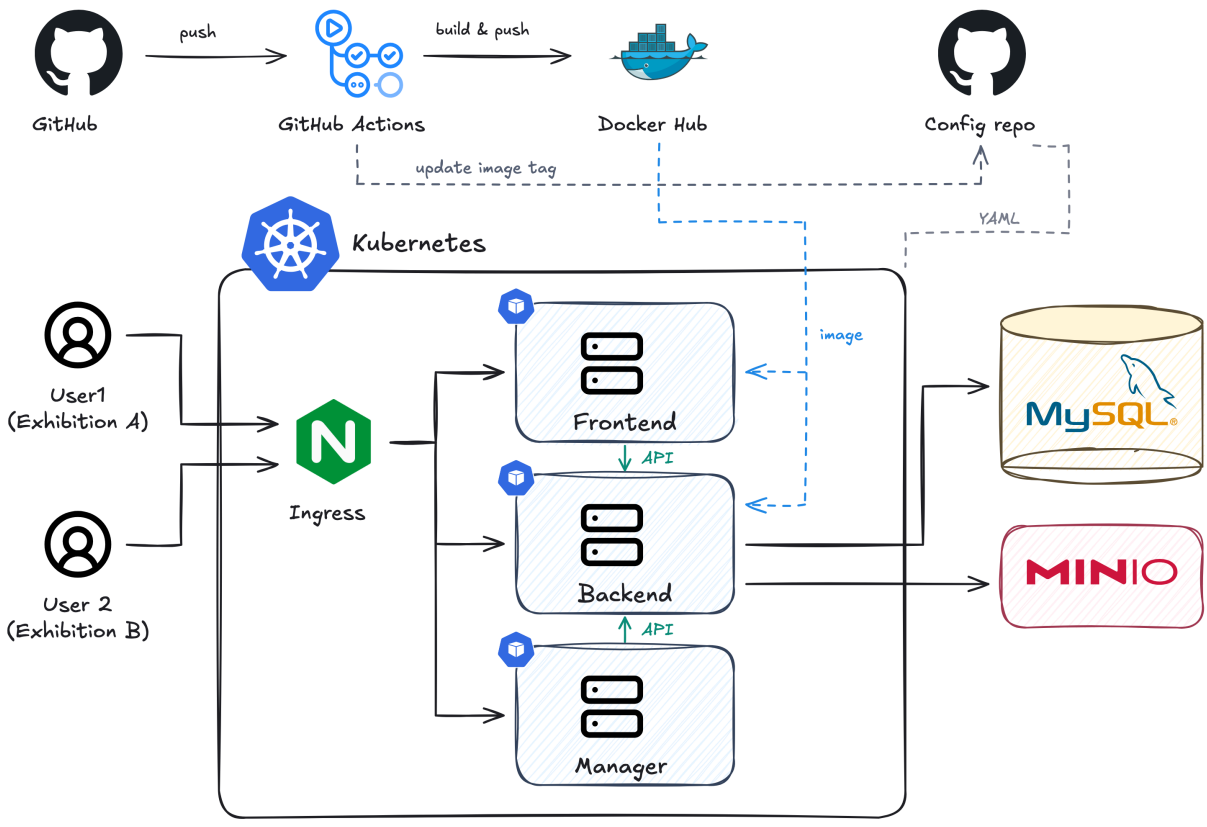
개발 동기

기존에는 학생들의 작품을 전시하는 사이트 3개를 따로 관리하고 있었습니다.

처음 하나의 전시 사이트를 완성한 뒤, 다른 종류의 전시 사이트 개설 요청이 들어오면서 기존 코드를 fork해 사용했습니다. 목적과 기능은 물론 코드도 거의 같았지만, 세 개의 소스코드를 따로 관리해야 했습니다.

개발한 지 오래된 사이트라 UI와 UX가 불편하다는 의견도 있었습니다. 유지보수 부담을 줄이고 사용성도 개선하기 위해, 멀티테넌시를 지원하는 통합 전시회 사이트를 만들기로 했습니다.

서비스 구조



통합 전시회 사이트의 서비스 구성과 이미지 배포 흐름

기존에는 두 학과에서 총 세 개의 전시 사이트를 각각 운영하고 있었습니다. 이 중 한 학과는 전시 목적에 따라 두 개의 사이트를 별도로 운영하고 있었지만, 세 사이트 모두 작품 조회나 카테고리 분류, 게시판 등 대부분의 기능은 비슷했습니다. 이를 하나로 통합하기 위해 Next.js 프론트엔드와 Spring Boot 백엔드를 공통으로 사용하고, 접속한 도메인에 따라 서로 다른 전시를 보여주는 구조로 만들었습니다.

각 도메인은 동일한 프론트엔드 서버로 연결됩니다. 프론트엔드는 서버에서 API를 요청할 때 접속 도메인을 Origin 헤더에 담아 전달하고, 백엔드는 이를 기준으로 전시를 식별합니다. 이후 해당 전시의 작품, 카테고리, 회차, 게시판 등의 데이터를 반환합니다. 덕분에 같은 코드를 사용하면서도 전시별로 콘텐츠와 설정을 다르게 적용할 수 있습니다.

관리자 기능은 별도의 매니저 사이트에서 제공합니다. 관리자 요청은 X-Exhibition-Id 헤더로 작업할 전시를 지정하고, 백엔드에서 해당 전시에 대한 관리 권한을 확인합니다. 전시 데이터와 파일 정보는 MySQL에 저장하며, 이미지와 영상은 MinIO에서 관리합니다. 미디어 파일을 조회할 때는 백엔드의 미디어 API를 통해 제공합니다.

주요 작업

- 두 학과에서 운영하던 세 개의 전시 사이트를 하나의 Next.js 프론트엔드와 Spring Boot 백엔드로 통합
- 도메인 기반 멀티테넌시 구현 및 전시별 데이터와 관리자 권한 분리
- 전시, 회차, 카테고리, 작품, 분류, 게시판 등의 API 개발
- 카카오 로그인, JWT 인증 및 작품 좋아요 기능 구현
- 전시별 화면 설정과 관리자 페이지를 통한 콘텐츠 관리 기능 구현
- Excel 기반 작품 일괄 등록 및 MinIO 미디어 업로드 및 스트리밍 기능 구현
- Spring REST Docs를 이용한 API 문서화 및 주요 기능 테스트 작성

구현 과정

기존 전시 사이트 통합

두 학과에서 운영하던 세 개의 전시 사이트는 목적과 관리하는 데이터가 달랐지만, 작품 조회나 카테고리 분류, 게시판 등 대부분의 기능은 비슷했습니다. 기존에는 사이트마다 코드를 별도로 관리하고 있어 같은 기능을 수정하더라도 각 사이트에 반복해서 반영해야 했습니다. 이를 해결하기 위해 공통 기능을 하나의 프론트엔드와 백엔드로 통합했습니다. 대신 전시별로 도메인과 데이터를 구분하고, 로고나 배너, 수상작 표시 여부, 기본 회차 등의 설정을 다르게 적용할 수 있도록 했습니다.

전시별 데이터와 관리자 권한 분리

여러 전시가 하나의 백엔드를 사용하면서 가장 중요했던 부분은 서로 다른 전시회의 데이터가 섞이지 않도록 하는 것이었습니다. 공개 API에서는 Origin 헤더를 통해 접속 도메인을 확인하고, 해당 도메인에 연결된 전시회를 식별하도록 했습니다. 조회된 전시 정보를 기준으로 작품이나 카테고리 등을 조회하기 때문에 같은 API를 호출하더라도 접속한 사이트에 따라 다른 데이터를 반환합니다. 관리자 기능은 전체 관리자와 전시별 관리자로 권한을 나눴습니다. 관리자가 X-Exhibition-Id 헤더로 전시를 선택하면, 백엔드에서 해당 전시를 관리할 권한이 있는지 확인한 뒤 요청을 처리합니다.

Excel을 이용한 작품 일괄 등록

전시회에는 매 회차마다 많은 작품이 등록됩니다. 작품마다 제목과 발표자 정보, 분류, 이미지, 영상 등을 하나씩 입력하기에는 시간이 많이 걸리기 때문에 Excel을 이용한 일괄 등록 기능을 구현했습니다. 관리자가 Excel 파일과 미디어 파일을 함께 업로드하면, 먼저 Excel의 필수 항목과 전시회 및 회차 정보를 검사합니다. 이후 Excel에 적힌 파일명과 실제 업로드된 파일을 비교하고, 파일 확장자와 시그니처를 확인한 뒤 작품과 미디어를 등록합니다. 대량 등록 과정에서 일부 파일의 업로드가 실패하거나 DB 트랜잭션이 롤백되는 상황도 고려했습니다. 이미 업로드된 MinIO 객체를 정리하도록 처리해 파일과 데이터베이스 사이에 불필요한 데이터가 남지 않도록 했습니다.

대량 업로드 처리 개선

여러 작품을 한 번에 등록할 때 불필요한 DB 조회와 파일 전송이 반복되지 않도록 처리 방식을 개선했습니다. Excel 안에서 동일한 카테고리나 분류가 반복되면 요청 내에서 조회 결과를 재사용하고, 작품과 분류의 연결 정보는 모아서 저장합니다. 또한 같은 미디어 파일을 여러 작품에서 참조하더라도 실제 파일 업로드는 한 번만 수행하도록 했습니다. MinIO로 파일을 전송하는 작업은 별도의 스레드 풀에서 병렬로 처리했습니다. 이때 DB 작업은 기존 트랜잭션 스레드에서 처리하고, 네트워크 I/O만 분리해 트랜잭션 관리에 영향을 주지 않도록 했습니다.

미디어 파일 스트리밍

전시 사이트에서는 이미지뿐 아니라 용량이 큰 발표 영상도 제공해야 했습니다. 파일 전체를 서버 메모리에 올려서 전달하면 메모리 사용량이 커질 수 있어 스트리밍 방식으로 구현했습니다. MinIO에서 파일 정보를 조회한 뒤 Spring의 Resource를 이용해 파일을 스트리밍하고, 응답에 Content-Type, 파일 크기, 캐시 관련 헤더를 포함하도록 했습니다. 이를 통해 파일 전체를 메모리에 적재하지 않고도 이미지와 영상을 제공할 수 있도록 했습니다.

장점

전시마다 도메인과 콘텐츠를 따로 관리하면서도 공통 API를 사용할 수 있습니다. 관리자 화면에서 엑셀과 미디어를 함께 등록하고, 작품 조회부터 영상 제공까지 같은 백엔드에서 처리할 수 있게 됐습니다.

관련 링크

- GitHub 저장소
- 성균관대학교 소프트웨어융합대학 소프트웨어학과 졸업작품 전시회

인프라

Kubernetes One Repo

온프레미스 Kubernetes 클러스터의 GitOps 구조를 설계하고, 모니터링과 로그 수집, IP 접근 제어를 추가했습니다.

개발 기간 2026.06 ~

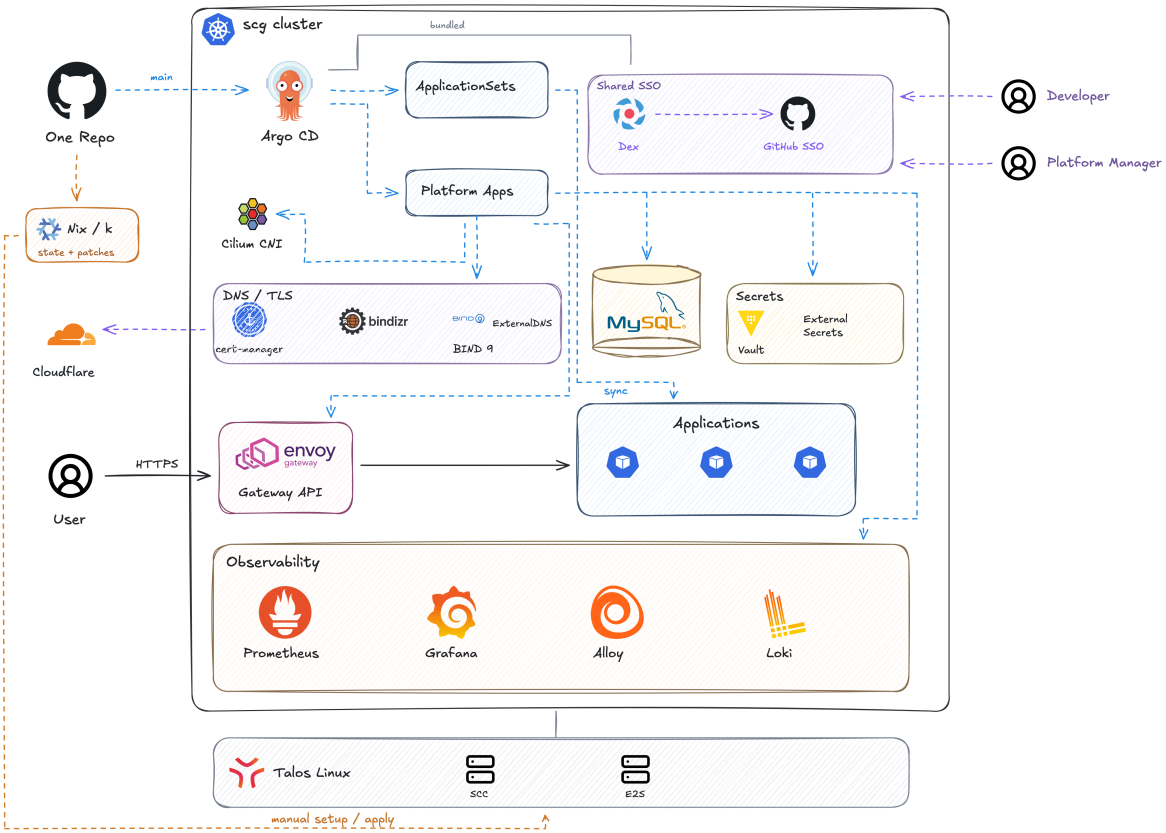
Kubernetes, Argo CD, Prometheus, Grafana, Loki, Alloy

저장소 보기 ↗

프로젝트 소개

성균관대학교 시스템컨설턴트그룹의 온프레미스 Kubernetes 클러스터를 관리하는 저장소입니다. 클러스터는 Talos Linux를 기반으로 하며, 플랫폼 설치와 애플리케이션 배포에 필요한 설정을 Git에서 관리합니다. Argo CD가 이 설정을 읽어 클러스터에 반영합니다.

아키텍처



Kubernetes One Repo 아키텍처

주요 작업

초기 GitOps 구조를 설계한 뒤, 클러스터 상태와 애플리케이션 로그를 확인할 수 있도록 모니터링과 로그 수집을 추가했습니다. 여러 서비스에서 사용할 Gateway와 IP 접근 제어도 함께 작업했습니다.

- `meta.yaml`과 ApplicationSet을 이용한 GitOps 구조 설계

- 공통 Gateway와 cert-manager 인증서 설정
- Prometheus와 Grafana를 이용한 클러스터 모니터링
- Alloy, Loki, Grafana를 연동한 애플리케이션 로그 수집과 조회
- 공통 Helm 차트에 CIDR 기반 IP 접근 제어와 배포 전 검증 추가

구현 과정

meta.yaml로 플랫폼 컴포넌트 등록

초기 구조에서는 각 플랫폼 컴포넌트에 `meta.yaml`을 두었습니다. ApplicationSet이 `platform/*/meta.yaml`을 읽어 Argo CD Application을 생성하고, 새 컴포넌트도 같은 방식으로 등록하도록 했습니다.

Kustomize로 관리하는 설정과 Helm 차트를 모두 등록할 수 있게 했습니다. Helm 차트는 저장소의 values 파일과 추가 매니페스트를 함께 적용할 수 있도록 했습니다.

AppProject와 ApplicationSet은 클러스터 루트에 모아두고, 생성된 Application에는 auto-sync, pruning, self-healing을 적용했습니다. Git에서 삭제한 리소스는 클러스터에서도 삭제하고, 클러스터에서 직접 수정한 설정은 Git에 저장된 상태로 되돌리도록 했습니다.

공통 Gateway와 HTTPS 설정

외부 요청을 Cilium Gateway에서 받도록 설정했습니다. HTTPS 리스너와 함께 cert-manager의 ClusterIssuer, Certificate 리소스도 Git에서 관리했습니다.

각 서비스의 Route는 공통 Gateway에 연결합니다. Route 연결을 허용할 네임스페이스는 라벨로 구분했습니다.

노드 모니터링과 Grafana 로그인 연동

kube-prometheus-stack을 Argo CD에 등록해 노드와 워크로드의 메트릭을 수집했습니다. Talos 환경에서 수집 가능한 메트릭에 맞춰 수집 대상과 alert rule을 조정했습니다. Grafana에는 CPU와 메모리 사용량, 네트워크와 디스크 상태를 한곳에서 볼 수 있는 대시보드를 만들었습니다.

Grafana는 Dex와 연동해 GitHub 계정으로 로그인하도록 했습니다. GitHub의 `platform` 팀원에게는 Admin, `active` 팀원에게는 Viewer 권한을 부여했습니다. 대시보드는 JSON 파일로 Git에서 관리하고, Grafana에서는 읽기 전용으로 제공했습니다.

Pod 라벨을 기준으로 로그 수집

각 노드에서 Alloy가 Pod 로그를 수집해 Loki로 보내고, Grafana에서 조회하도록 연결했습니다. 메트릭과 로그는 Grafana에서 함께 확인하되, Loki와 Alloy는 모니터링과 별도의 Argo CD Application으로 관리했습니다.

로그 수집 대상은 앱 이름과 실행 환경을 나타내는 Pod 라벨로 구분했습니다. 공통 Helm 차트는 이 라벨을 기본으로 붙이고, Kustomize로 배포하는 앱은 Pod 템플릿에 같은 라벨을 추가하면 됩니다.

수집한 로그는 앱과 실행 환경뿐 아니라 워크로드, Pod, 컨테이너별로 검색할 수 있도록 라벨을 정리했습니다. 로그 보관 기간과 저장 공간도 설정하고, Alloy의 RBAC와 Loki 접근 정책을 함께 관리했습니다.

Helm 차트에 IP 접근 제어와 설정 검증 추가

운영 서비스에 접근할 수 있는 IP 대역을 제한하기 위해 공통 Helm 차트에 `allowCIDRs`를 추가했습니다. 허용할 대역을 적으면 차트가 HTTPRoute의 라우팅 규칙에 맞는 Envoy SecurityPolicy를 생성합니다.

하나의 라우팅 규칙에 여러 백엔드나 트래픽 미러링 대상이 연결되면, 허용 IP 목록이 서로 같은지 검사합니다. 정책을 적용할 규칙 이름이 중복될 때도 차트 렌더링을 중단해 충돌하는 설정이 배포되지 않도록 했습니다.

모니터링과 로그 수집 설정에도 배포 전 검증을 추가했습니다. Helm 차트를 렌더링한 결과에서 수집 대상, 로그 보관 설정, Grafana 권한 매핑을 확인합니다. Prometheus나 Loki처럼 내부에서만 사용하는 서비스가 외부에 노출되지 않는지도 검사합니다.

관련 링크

- GitHub 저장소
- 초기 GitOps 플랫폼 구성
- 모니터링 플랫폼 구성
- 중앙 로그 수집 구성
- 공통 차트의 CIDR 접근 제어

개발 도구

pi-provider-manager

Pi Agent에서 OpenAI-Compatible Provider를 쉽게 관리할 수 있는 Pi Extension

개발 기간 2026.08

TypeScript, Pi Agent

저장소 보기 ↗

프로젝트 소개

Pi Agent에서 OpenAI-Compatible Provider를 쉽게 관리할 수 있는 Pi Extension입니다.

개발 동기

기존에 Pi Agent에서 로컬 LLM이나 사설 게이트웨이를 이용하려면 직접 models.json을 수정해야 하는 불편함이 있었습니다. 또한 해당 게이트웨이에서 사용 가능한 모델 카탈로그가 업데이트되어도 에이전트에 바로 반영되지 않는 불편함이 있었습니다.

주요 기능

- Provider 추가, 조회, 수정, 삭제
- Provider별 모델 목록 새로고침
- 모델별 컨텍스트 크기와 최대 출력 토큰 수 설정
- Provider 설정과 API 키를 별도 파일에 저장

장점

기존처럼 직접 json 파일을 수정하지 않고 에이전트 TUI 내부에서 쉽게 Provider를 관리할 수 있게 되었습니다. 모델 카탈로그 업데이트나 API 키 업데이트의 경우에도 에이전트 TUI 에서 바로 업데이트 할 수 있습니다.

설치

```
pi install npm:@2tle/pi-provider-manager
```

관련 링크

- GitHub 저장소
- npm 패키지

백엔드

S-TOP

성균관대학교 Cyber S-TOP 사이트의 백엔드 개발을 담당했습니다.

개발 기간 2024

Java, Spring Boot, MySQL

프로젝트 소개

S-TOP이란?

- 성균관대학교 소프트웨어융합대학에서 주최하는 온라인 행사
- 소프트웨어융합대학 학생들이 만든 우수 작품 전시
- 기업 대표 등의 강의 시청
- 기업의 채용 정보를 제공
- 기업의 산학협력 프로젝트 문의와 과제 제안 접수
- 투표 및 퀴즈 이벤트 운영

사용 기술 스택

- 프론트엔드 → TypeScript / Next.js
- 백엔드 → Java / Spring Boot
- 데이터베이스 → MySQL / Notion

작업 내역

- 스타트업 대담 영상 및 잡페어 인터뷰 CRUD 구현
- 대담 영상의 퀴즈 CRUD 및 채점 기능 구현
- 관리자용 대담 영상의 퀴즈 제출 결과 Excel 다운로드 기능 구현

관리자용 대담 영상의 퀴즈 제출 결과 Excel 다운로드 기능 구현

문제 상황

- 관리자는 대담 영상의 퀴즈 제출 결과를 엑셀로 받아야 한다.
- 엑셀에는 학생 정보와 퀴즈 응시 내역이 포함되어야 한다.

구현 방법

1. 퀴즈 결과만 변환할 수 있는 엑셀 변환 코드를 작성한다.
    - 오직 퀴즈 데이터만 엑셀로 변환할 수 있음.
    - Java Reflection을 사용하지 않아 변환이 빠름
2. **여러 데이터 구조에 대응할 수 있는 공통 엑셀 변환 유틸리티를 만든다.**
    - 추후 다른 엑셀 변환 기능이 생길 수 있음
    - 공통 모듈로 만들어 코드 재사용성 증가
    - Java Reflection 사용

다른 기능에서도 엑셀 변환이 필요할 수 있어 2안을 선택함.

개발 과정

1. `@ExcelColumn`과 `@ExcelDownload` 어노테이션 추가
    - `ExcelColumn`
        - 엑셀의 각 열을 정의
        - 데이터 클래스의 필드에 붙임
        - 각 열의 이름을 지정할 수 있음
    - `ExcelDownload`
        - 엑셀로 변환할 수 있는 클래스임을 표시
        - 생성할 시트의 이름을 지정할 수 있음
        - 생성할 엑셀 파일명을 지정할 수 있음
2. `ExcelUtil` 유틸리티 클래스 제작
    - 클래스와 필드의 어노테이션에서 파일명, 시트명, 헤더명을 읽어옴
    - 헤더 목록을 만들고 `SXSSFWorkbook`에 작성
    - 데이터 목록을 순회하며 행을 만들고 값을 채움
    - 엑셀 유틸리티를 Bean으로 등록해 다른 Bean에서 주입받아 사용

총평

- `SXSSFWorkbook`으로 엑셀을 생성해 메모리를 효율적으로 사용
- 이후 다른 파트의 엑셀 변환 기능이 없어짐
- 결과적으로 오버 엔지니어링이 되었음

개선 과정

로그인을 해도 되고, 안 해도 되는 API가 있다.

문제 상황

- 기존 인증/인가 방식에서는 로그인 필수 API와 비로그인 API만 구분할 수 있었음
- 비로그인 상태에서도 GET 요청이 가능해야 한다.
- 로그인 상태에서는 사용자에 맞게 데이터를 가공해서 제공해야 한다.

해결 과정

- 기존 유저 권한 검사 로직에 Optional을 추가함
- 권한이 Optional인 API에 토큰 없이 요청하면 사용자 정보 대신 null을 전달
- 토큰이 있다면 Optional이어도 기존 토큰 검증과 권한 검사를 수행

프론트엔드에서 Optional 인가 방식이 작동하지 않는다.

문제 상황

- 앞서 만든 Optional 인가 방식이 제대로 동작하지 않음
- 프론트엔드에서 Optional 인가 방식을 쓰는 API 요청 시 리프레시 토큰 오류가 발생함

원인

- 프론트엔드에서 비로그인 상태로 API를 호출하면 `Authorization: Bearer null`이 전달됨
- 백엔드는 비로그인 요청에 Authorization 헤더가 없을 것으로 가정하고 개발함
- JWT 검사에서 인증 방식이 Optional이고 Authorization 헤더가 없는 경우에만 검증을 건너뛰고 있었음

해결 과정

- JWT 인증/인가 코드에서 토큰 여부를 확인하는 메서드를 수정
- Authorization 헤더가 없거나 값이 `Bearer null`이면 토큰이 없는 것으로 판단해 false를 반환

백엔드

돌핀 (dolpin)



개발자를 위한 아티클 큐레이션 서비스입니다. 백엔드와 크롤러 개발을 담당했습니다.

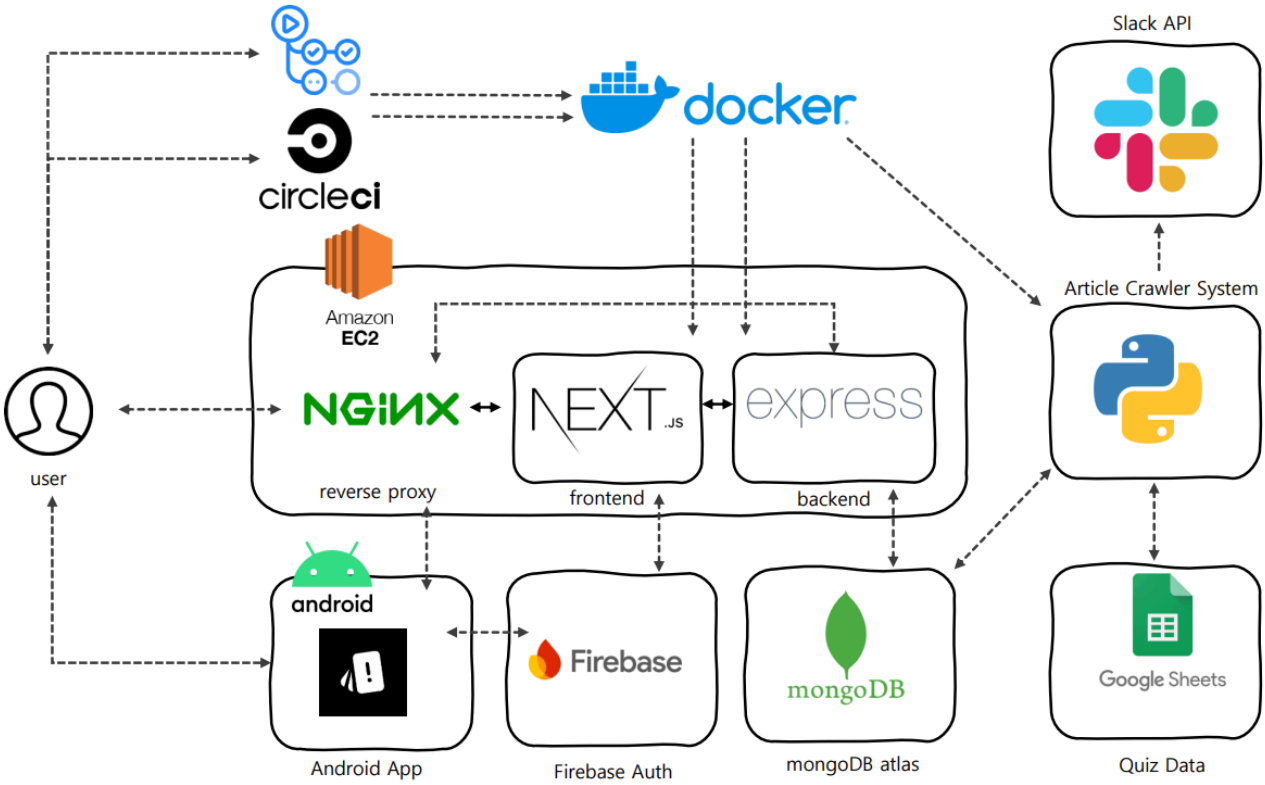
개발 기간 2023 ~ 2024

TypeScript, Express, MongoDB, Python, Docker

프로젝트 소개

돌핀은 IT 개발자들을 위한 아티클 큐레이션 서비스입니다. 다양한 분야의 전문적인 글과 최신 IT 소식을 한 번에 확인할 수 있도록 만들었습니다. 당시 안드로이드 앱과 웹으로 베타 서비스를 운영했고, 저는 백엔드와 크롤러 개발을 담당했습니다.

서비스 구조



돌핀 서비스 구조

주요 작업

유저가 아티클을 조회하거나 공유하거나 북마크하면 가중치를 높이고, 오래된 아티클일수록 가중치를 낮추는 시스템을 개발했습니다. 유저에게는 가중치가 높은 아티클부터 보여주도록 했습니다. 서비스를 운영하면서 API 응답속도와 크롤러의 문제를 고치고, 배포도 자동화했습니다.

서버 응답이 느리고 타임아웃이 자주 발생한다

아티클이 많아지면서 목록을 한 번 조회하는 데 약 0.8 ~ 2초가 걸렸고, 로그인 과정에서도 타임아웃이 자주 발생했습니다.

처음에는 Mongoose의 `populate`를 의심해 `aggregate`로 바꿔봤습니다. 하지만 차이는 0.00x초 정도로 응답속도는 거의 나아지지 않았고, 타임아웃도 그대로였습니다.

다음으로 서버의 위치와 슬립 동작을 살펴봤습니다. Heroku는 유럽 리전, MongoDB는 한국 리전에 있어 요청할 때마다 데이터가 두 리전을 오갔습니다. Heroku는 약 30분 동안 요청이 없으면 슬립 상태로 전환돼, 백엔드가 다시 실행되기까지 시간도 걸렸습니다.

슬립 동작이 타임아웃의 원인일 것으로 추정해 백엔드를 AWS로 이전했습니다. 이전 후 테스트에서 API 응답속도가 개선됐고, 타임아웃 문제도 해결됐습니다.



AWS 이전 전 API 응답 기록



AWS 이전 후 API 응답 기록

하나의 기사를 여러 카테고리에 함께 보여주고 싶다

기존에는 `category`가 `number` 타입이라 한 아티클에 카테고리를 하나만 지정할 수 있었습니다. 여러 카테고리에 해당하는 글도 있어, 필드를 `categories`로 바꾸고 타입도 `array<number>`로 변경했습니다.

기존 문서와 새 문서가 섞여 있어 조회할 때는 `aggregate`를 사용했습니다. `category`만 있는 문서는 `[category]`를 담은 `categories` 필드를 만들고 기존 필드는 결과값에서 삭제했습니다. 이미 `categories`가 있으면 그대로 유지했습니다.

크롤러에는 제목의 키워드로 카테고리를 판별하는 함수를 추가해, 한 아티클을 관련된 여러 카테고리에서 보여주도록 했습니다.

크롤러와 배포 과정에서 불편했던 점을 고쳤다

크롤러는 수집 시간이 오래 걸리고 램 사용량도 높았습니다. import할 때 테스트용 함수까지 실행되는 코드를 찾아 정리했고, 각 크롤러에서 따로 만들던 웹드라이버는 메인 함수에서 한 번만 만들어 인자로 전달하도록 바꿨습니다.

개인 AWS 계정에 따로 배포하던 프론트엔드와 백엔드는 팀 공용 계정으로 옮기고, Docker와 GitHub Actions로 배포를 자동화했습니다. 크롤러는 클라우드 비용 부담 때문에 저전력 컴퓨터에서 실행했습니다. 학교 보안정책상 외부 접속이 막혀 있어 CircleCI로 빌드한 이미지를 Docker Hub에 올리고, `Crontab`으로 매일 2시에 pull해 크롤러를 돌렸습니다.

수집이 2주 넘게 멈췄는데도 알아채지 못한 적이 있어 슬랙 알림도 추가했습니다. `DeployBot`이 `#deploy` 채널로 크롤러의 시작과 종료, 에러 로그를 보내도록 했고, 팀원들은 슬랙에서 오류를 확인하고 해당 크롤러를 수정할 수 있게 됐습니다.